

Exam Date & Time: 04-Dec-2024 (02:30 PM - 05:30 PM)



MANIPAL ACADEMY OF HIGHER EDUCATION

MIT BLR - BTech VII Semester - End Semester Examination - Nov / Dec -2024

SOFTWARE TESTING [NPT-4003]

Marks: 50

Duration: 180 mins.

PART-A

Answer all the questions.

- 1) List different kinds of Dynamic Quality attributes with brief description. (3)
 - A)
 - B) Identify the limitations of Software Testing. (2)
 - C) For the given program, draw the data-flow graph, list the Definitions, P-Use and C-Use of all the variables used and list the DU pairs. Also give test cases to cover All-Definitions. (5)

```
void main()
{
float A,B,C;
printf("Enter number 1, number 2 and number 3:n");
scanf("%f%f%f", &A,&B,&C);
if(A>B) {
if(A>C) {
printf("The largest number is: %fn",A);
}
else {
printf("The largest number is: %fn",C);
}
}
else {
if(C>B) {
printf("The largest number is: %fn",C);
}
else {
printf("The largest number is: %fn",B);
}
}
}
}
```
- 2) A mobile phone service provider uses a program that computes the monthly bill of customers as follows: (4)

Minimum Rs 300 for up to 120 calls

- A) 300 Plus Re 1 per call for the next 70 calls
 300 Plus Rs 0.80 per call for the next 50 calls
 300 Plus Rs 0.40 per call for any call beyond 220 calls.
 Identify the test cases for weak normal and weak robust equivalence class testing.

- B) Consider the following program:

```
main ( )
{
int a,b,c, sum, diff, mul;
scanf( "%d %d %d", &a, &b, &c);
sum=calsum(a,b,c);
diff=caldiff(a,b,c);
mul=calmul(a,b,c);
printf("%d %d %d", sum,diff,mul);
}
calsum(int x, int y, in z){
int d;
d=x+y+z; return(d); }
```

(3)

Consider the following two scenarios.

- Suppose, the main () module is not ready for the testing of calsum () module. Apply suitable testing strategy to test the whole system in such a scenario.
- Suppose modules caldiff() and calmul() are not ready when called in main().Apply suitable testing strategy to test the whole system in such a scenario.

- C) Draw control flow graph for the following code.

```
a:=read()
b:=read()
c:=read();
a= a+b+c+d;
if(a< 10){
d=c+8;
write(c);
}
else if(a< 10)
{
e=10;
d=e+a;
write (e);
}
else {
f=12;
d=f+a;
write(f);
}
write (d);
```

(2)

Define test cases to achieve maximum coverage of branch and condition coverage.

- 3) List different coupling data flow coverage criteria.

(2)

- A)

B) Analyse different OO anomalies in object-oriented software. (3)

C) Consider the following program.

```
public class TrashAndTakeOut
{
    public static void main (String []argv)
    {
        int x = 0;
        x = Integer.parseInt (argv[0]);
        trash (x);
    }
    public static void trash (int x)
    {
        int m, n;
        m = 0;
        if (x > 0)
            m = 4;
        if (x > 5)
            n = 3*m;
        else
            n = 4*m;
        int o = takeOut (m, n);
        System.out.println ("o is: " + o);
    }
    public static int takeOut (int a, int b)
    {
        int d, e;

        d = 42*a;
        if (a > 0)
            e = 2*b+d;
        else
            e = b+d;
        return (e);
    }
}
```

(5)

(i) Identify Last Defs and First Uses of different variables.

(ii) Identify Coupling du-pairs in triplet form (method name, variable name, statement number)

4) Analyse different processes for regression test selection.

(3)

A)

B) A university is admitting students in a professional course subject to the following conditions: (5)

(a) Marks in Java ≥ 70

(b) Marks in C++ ≥ 60

(c) Marks in OOAD ≥ 60

(d) Total in all three subjects ≥ 220 OR Total in Java and C++ ≥ 150

If the aggregate mark of an eligible candidate is more than 240, he will be eligible for a scholarship course, otherwise he will be eligible for normal course.

The program reads the marks in the three subjects and generates the following outputs:

(i) Not eligible

(ii) Eligible for scholarship course

(iii) Eligible for normal course

Design test cases for this program using decision table testing.

C) List different formal models to work with test coverage criteria with brief description. (2)

5) For the given requirement R and program P, enhance the following test set T using the mutation operator M.

R: The program should print biggest of 3 numbers a,b,c. If the numbers are equal, it should print the first number in the sequence to be the biggest.

A)

P:

```
void CompareThreeNos( )
{ int a,b,c; input( a,b,c);
  if((a>=b) and (a<=c))
  { print( a is biggest);}
  else if((b>=a) and (b>=c))
  { print( b is biggest);}
  else{ print( c is biggest);}
}
```

(5)

TC# a b c Exp O/P

1 0 0 0 a is biggest

2 3 1 2 a is biggest

M: Logical AND Operator is changed to OR.

B) Consider a three-input program to handle personal loans of a customer. Its input is a triple of positive integers (say principal, rate and term).

1000 <= principal <=40000

1<=rate<=18

1<=term<= 6

The program should calculate the interest for the whole term of the loan and the total amount of the personal loan. (5)

The output is:

interest = principal * (rate/100) * term

total_amount = principal + interest

Generate normal and robust boundary value test cases.

-----End-----